2023.1 Multicore Computing

Final Exam (7pm-8:20pm, June 16)

StudentID# : ( ) , Name : ( )

*** Your answer should be written in English.**

1. (21 points) Fill in the blanks (a)~(g) with the most appropriate English words.

- (a.) is a proprietary compiler by Nvidia intended for use with CUDA. CUDA code runs on both the CPU and GPU. (same as (a)) separates these two parts and sends host code (the part of code which will be run on the CPU) to a C/C++ compiler like GCC or Intel C++ Compiler (ICC) or Microsoft Visual C++ Compiler, and sends the device code (the part which will run on the GPU) to the GPU. The device code is further compiled by (same as (a)).
- (b.) is a C++ template library for CUDA based on the Standard Template Library (STL). (same as (b)) allows you to implement high performance parallel applications with minimal programming effort through a high-level interface that is fully interoperable with CUDA C. This means (same as (b)) is a library for high-level parallel algorithms and data structures that utilize GPU based on CUDA. It is a parallel analog of C++ STL and provides a productive way to program CUDA.
- The name of the OpenMP library function that returns the current thread index is (c.).
- In CUDA device, threads within the same (d.) can access the same data in a (e.) memory which is much faster than global memory.
- (f.) is a type of OpenMP schedule that is similar to the dynamic schedule, but the size of the portion of work decreases as the program runs.
- [In OpenMP] By default, all variables declared outside a parallel block are shared variables except (g.).

2. (16 points) Following program intends to compute the approximation value of numerical integration $\int_0^1 \frac{4.0}{(1+x^2)} dx$ with multiple number of threads (i.e. 8 threads) using OpenMP.

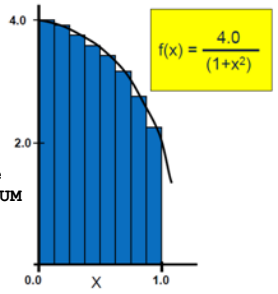
(1) Insert in line 11 appropriate code that specifies the number of threads to use in subsequent parallel regions to be **NUM.**

(2-a) The following program is not correct and may generate a wrong result. Which part in the program is wrong? : line ( )

(2-b) Why is the program wrong? Your answer : **([use less than 20 English words])**

(2-c) How can you modify the program code for correction? **Modify or insert your code directly in the following code.**

```
1:  #include <omp.h>
2:  #include <stdio.h>
3:  #define NUM 8
4:  long num_steps = 10000000;
5:  double step;
6:
7:  void main ()
8:  {
9:        long i; double x, pi, sum = 0.0;
10:       step = 1.0/(double) num_steps;
11:       [                                ]  // specifies the number of threads to use
                                               // in subsequent parallel regions to be NUM
12:       #pragma omp parallel for reduction(+:sum)
13:       for (i=0;i< num_steps; i++){
14:                 x = (i+0.5)*step;
15:                 sum = sum + 4.0/(1.0+x*x);
16:       }
17:       pi = step * sum;
18:       printf("pi=%.4lf\n",pi);
19: }
```



3. (16 points) (a) What is the meaning of the OpenMP directive "#pragma omp parallel sections"?

([use less than 15 English words])

(b) What is the main similarity between private variables and firstprivate variables in OpenMP?

([use less than 15 English words])

(c) What is the main difference between private variables and firstprivate variables in OpenMP?

(i) private: **([use less than 10 English words])**

(ii) firstprivate: **([use less than 10 English words])**

4. (28 points) Consider applying a 1D stencil to a 1D input array with an arbitrary size. Each output element is the sum of input elements within a radius. If radius = 3, then each output element is the sum of 7 elements. Elements that are not available because their indices are below 0 or above the size of an array are assumed to be 0. For example, if an input array is {3,1,6,2,4,0,5,1,7,8} and the radius is 2, the output array becomes {10,12,16,13,17,12,17,21,21,16}. Write an efficient CUDA kernel function **stencil1D** in the box (a) that can handle vectors with arbitrary size '**vec_size**'. Assume **vec_size**<2,000,000. Insert appropriate codes into the boxes (b), (c), (e) for memory management, and the box (d) for CUDA kernel function call. Assume that the number of threads per block is **THREAD_NUM** (i.e. 128). For **stencil1D** function implementation, using only global memory is OK. This means you do not need to use shared memory. Your code should be grammatically and logically correct, and handle correct memory management.

```
#include <stdio.h>
#include <stdlib.h>
#define THREAD_NUM 128  // CUDA kernel should generate 128 threads per block

__global__ void stencil1D(int *a, int *c, int vec_size, int radius) {
  (a)
}

int main(void) {
       int N, Radius, *a, *c, *d_a, *d_c;
       printf("vector size, and radius of stencil :");
       scanf("%d %d",&N, &Radius); // get the size of vectors as a keyboard input
       (b)
       a = (int *)malloc(N*sizeof(int)); vector_init(a, N);
       c = (int *)malloc(N*sizeof(int));
       (c)
       stencil1D (d)
       (e)
       for (int i=0;i<N;i++) printf("a[%d]=%d , c[%d]=%d\n",i,a[i],i,c[i]);
       free(a); free(c); cudaFree(d_a); cudaFree(d_c);
       return 0;
}
```

```
void vector_init(int* x, int size)
{
       int i;
       for (i=0;i<size;i++) {
              x[i]=i;
       }
}
```

Example of Execution Output Result:

```
vector size, and radius of stencil :
1234567 , 3   <---- user input

a[0]=0 , c[0]=6
a[1]=1 , c[1]=10
a[2]=2 , c[2]=15
a[3]=3 , c[3]=21
a[4]=4 , c[4]=28
a[5]=5 , c[5]=35
a[6]=6 , c[6]=42
a[7]=7 , c[7]=49
a[8]=8 , c[8]=56
a[9]=9 , c[9]=63
a[10]=10 , c[10]=70

...

a[1234566]=1234566 , c[1234566]=4938258
```

(f) Justify your answer for the empty box (d). (Explain in English how you derived your answer for (d).)

((f))

5. (19 points) Following pseudocode, which we learned in our class, describes parallel randomized quicksort algorithm **Parallel_QuickSort** using a divide-and-conquer approach. Assume that the function **Parallel_Partition** correctly defines the parallel partitioning algorithm. **spawn** means creating and starting a new thread. Assume that we use a typical multicore computer.

(1) Although the following algorithm **Parallel_QuickSort** is OK in theory, **Parallel_Quicksort** may have a serious problem in practice when implemented into real code and executed as is described in the algorithm (especially when input array is very large). What can be the serious problem? Explain. Your answer should include why such problem can be caused.

(i) the serious problem : ([use less than 10 English words])

(ii) why?: ([use less than 20 English words])

```
Parallel_QuickSort ( A[q : r] ) // sort the elements in A[q : r] using parallel randomized quicksort algorithm
 1. if q < r then
 2.     select a random element x from A[q : r]
 3.     k <- Parallel_Partition (A[q : r], x)
 4.     spawn Parallel_QuickSort ( A[q : k − 1] )
 5.           Parallel_QuickSort ( A[k + 1 : r] )
 6.     sync
```

(2) How can you modify the algorithm to solve the problem? Modify or insert your code directly in the above pseudocode.